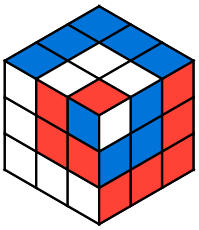
The "magic-cubes" package

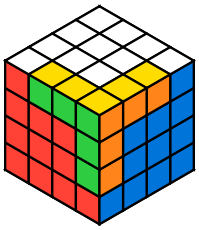
v0.1.0 MIT

Represent Rubik's cubes of arbitrary size and apply algorithms to them

Rodrigo Alcántara
@RodAlc24



U' L' U' F' R2 B'
R F U B2 U B' L
U' F U R F'



F U2 L F L' B L
U B' R' L' U R' D'
F' B R2

Table of Contents

Part I

Introduction

In 1974, Ernő Rubik invented a mechanical puzzle and called it the *Magic Cube*. Years later, in 1980, the puzzle was renamed the *Rubik's Cube*, the name by which it is now known. Today, it is considered to be the world's bestselling puzzle game.

MAGIC-CUBES is a package built on top of CeTZ that allows you to create, manipulate, and render Rubik's cubes of any size.

I.1 Terminology

Algorithm A sequence of moves written using standard cube notation.

Face One of the six flat sides of the cube. In a solved cube, all stickers on a face have the same color.

Layer A physical slice of the cube. It can contain a face (depth equal to 1) or not (depth greater than 1).

Depth The position of a layer measured from a given face. The outermost layer has depth 1.

Size (of a cube) The number of stickers on each edge. A standard 3x3x3 cube has a size of 3.

Sticker One of the colored pieces on the cube. Each face on a 3x3x3 cube has 9 stickers.

> All code of this package was written by a human, no AI tools were used.
>
> The documentation was also written by a human, and reviewed by AI.

Part II

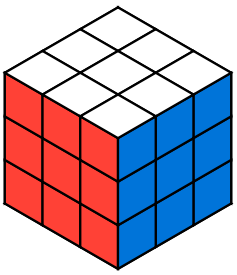
Quick Start

To start using MAGIC-CUBES, add the following import at the top of your `.typ` file:

```
#import "@preview/magic-cubes:0.1.0": *
```

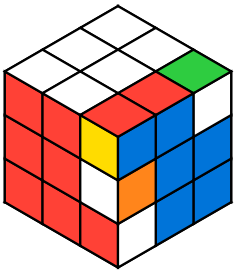
Creating and rendering a solved cube only requires two functions:

```
#draw_cube(cube())
```



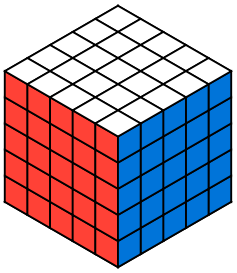
You can apply an algorithm before rendering the cube:

```
#draw_cube(
  apply(
    cube(),
    "R U R' U'"
  )
)
```



The package supports cubes of arbitrary size:

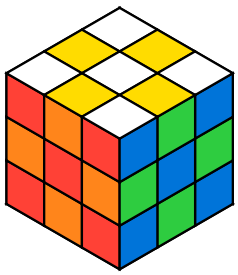
```
#draw_cube(
  cube(size: 5)
)
```



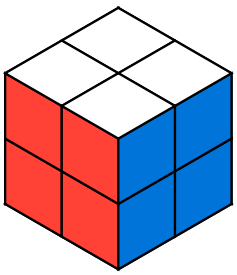
Part III

Examples

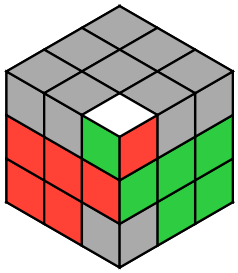
```
#draw_cube(
  apply(
    cube(),
    "M2 E2 S2"
  )
)
```



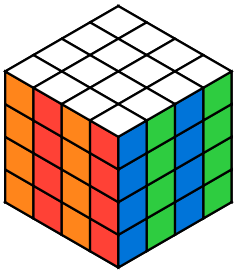
```
#draw_cube(
  cube(size: 2)
)
```



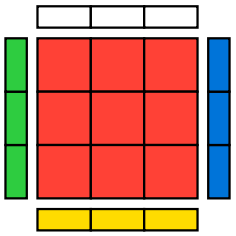
```
#draw_cube(
  apply(
    f2l-cube,
    "R2 U R2 U R2 U2
R2",
    inverse: true,
  )
)
```



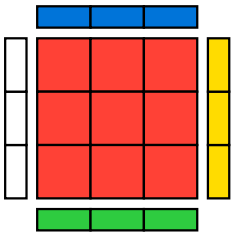
```
#draw_cube(
  apply(
    cube(size: 4),
    "R2 3R2 F2 3F2 R2
3R2"
  )
)
```



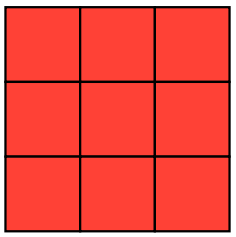
```
#draw_face(
  cube(),
  "f"
)
```



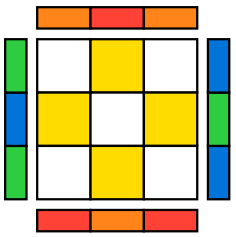
```
#draw_face(
  cube(),
  "f",
  top-face: "r"
)
```



```
#draw_face(
  cube(),
  "f",
  length: 84pt,
  adjacent-faces:
false
)
```

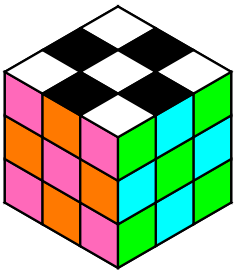


```
#draw_face(
  apply(
    cube(),
    "M2 E2 S2"
  ),
  "u",
)
```

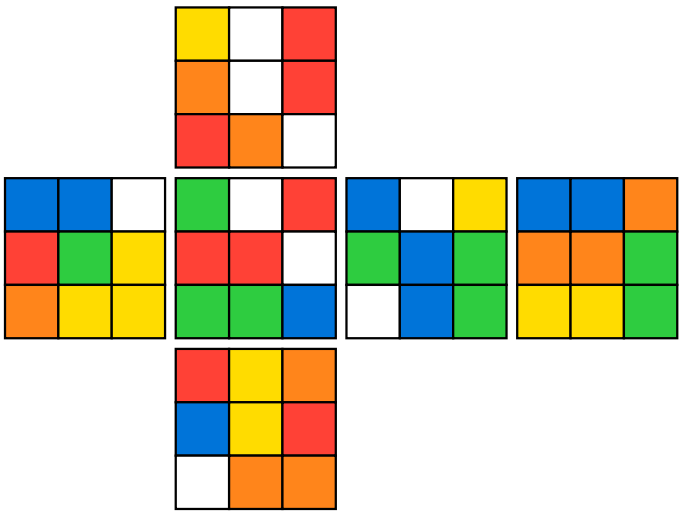


III Examples

```
#draw_cube(
  apply(
    cube(
      colors: (
        f: rgb("#ff69ba"),
        r: rgb("#00ff00"),
        u: rgb("#ffffff"),
        b: rgb("#ff7900"),
        l: rgb("#00ffff"),
        d: rgb("#000000"),
      )
    ),
    "M2 E2 S2"
  )
)
```



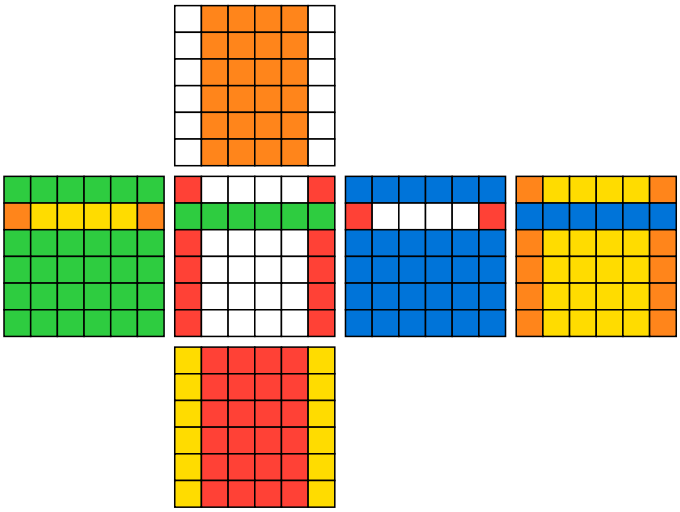
```
#draw_flat(
  rotate_cube(
    apply(
      cube(),
      "f r b u r f r u d b l f r l u r l u f "
    ),
  "z",
  )
),
```



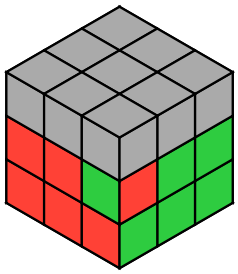
,

III Examples

```
#draw_flat(
  apply(
    cube(
      size: 6
    ),
    inverse: true,
    "2U 2-5r"
  ),
)
```

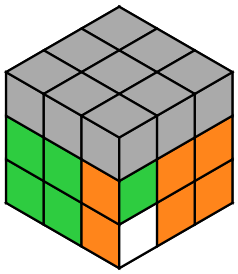


```
#draw_f2l(
  "(R U2 R' U) (R U2 R' U) y' (R' U' R) y"
)
```



(R U2 R' U) (R U2 R' U) y' (R' U' R) y

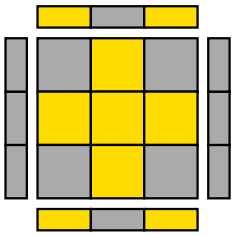
```
#draw_f2l(
  "(R U' R' U) d (R' U' R U') (R' U R)"
)
```



(R U' R' U) d (R' U' R U') (R' U R)

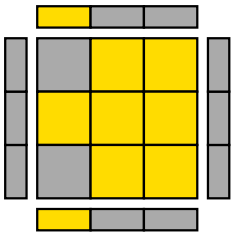
III Examples

```
#draw_oll(
  "(R U2 R' U') (R U R' U') (R U' R')"
)
```



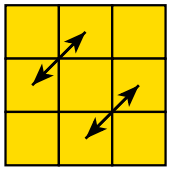
(R U2 R' U') (R U R' U') (R U' R')

```
#draw_oll(
  "(r U R' U') (r' F R F')"
)
```



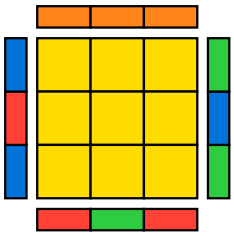
(r U R' U') (r' F R F')

```
#draw_pll(
  "(M2 U M2 U) (M' U2) (M2 U2) (M' U2)"
)
```



(M2 U M2 U) (M' U2) (M2 U2) (M' U2)

```
#draw_pll(
  "R U' R U R U R U' R' U' R2",
  adjacent-faces: true,
  arrows: false
)
```



R U' R U R U R U' R' U' R2

Part IV
User Guide

IV.1 Importing the package

To start using MAGIC-CUBES, add the following import at the top of your `.typ` file:

```
#import "@preview/magic-cubes:0.1.0": *
```

IV.2 Creating cubes

A cube is represented internally as a dictionary, and can be easily created with the `#cube` function. It should not be created or modified directly.

The size of the cube can be specified with the `⟨size⟩` argument (default is 3). The face colors can be changed with the `⟨colors⟩` argument, specifying the faces to change. The default face colors are:

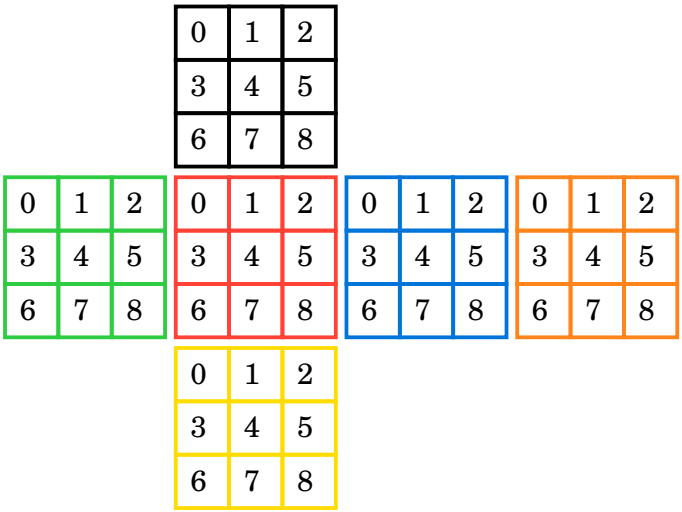
```
(
  f: red,
  r: blue,
  u: white,
  b: orange,
  l: green,
  d: yellow
)
```

These colors are the Typst predefined colors[1].

It is also possible to specify the stickers of a face manually, in that case the corresponding value in `⟨colors⟩` is ignored. When specifying the colors manually, all the stickers in the face must be specified.

The order of the stickers in a face is the following:

[1] https://typst.app/docs/reference/visualize/color/#predefined-colors

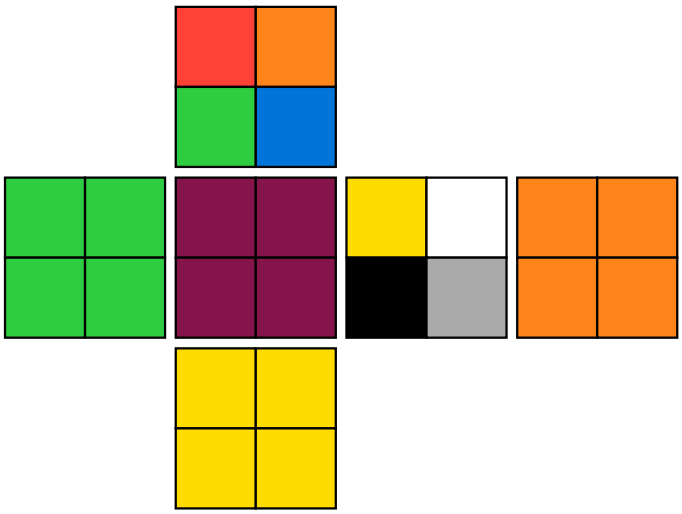


Each face is shown using its default color described above, except for the upper face, which is shown in black.

Note that this way of creating cubes may result in an impossible cube state (you can even use more than six colors!). This can be useful, for example, to add a gray color to the non-relevant pieces.

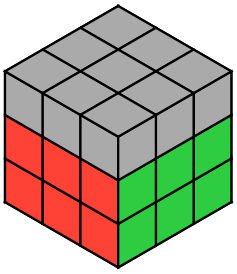
A similar order is used in larger or smaller cubes, for example, if we want to create a 2x2x2 cube with a custom color on the front face and custom upper and right faces we can do:

```
#draw_flat(
  cube(
    size: 2,
    colors: (f: maroon),
    stickers: (
      u: (
        red,
        orange,
        green,
        blue,
      ),
      r: (
        yellow,
        white,
        black,
        gray,
      ),
    )
  )
)
```

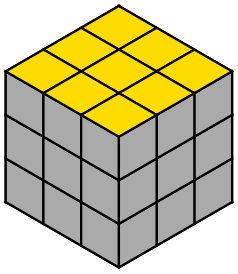


There are also some useful predefined cubes: `#f2l-cube` and `#oll-cube`.

```
#draw_cube(f2l-cube)
```



```
#draw_cube(oll-cube)
```

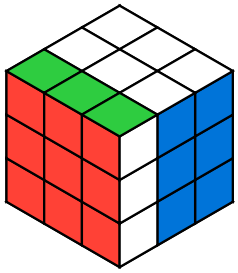


IV.3 Applying algorithms

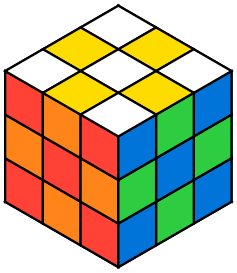
One of the main features of MAGIC-CUBES is the ability to apply algorithms to the cubes. Algorithms are written following the standard notation that is fully documented in Section V.

The function `#apply` takes a cube and an algorithm string. It is also possible, using the `⟨inverse⟩` argument to apply the inverse algorithm. This results in a cube that returns to the original state after applying the specified algorithm.

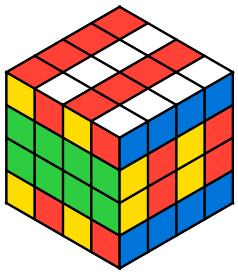
```
#draw_cube(
  apply(
    cube(),
    "F"
  )
)
```



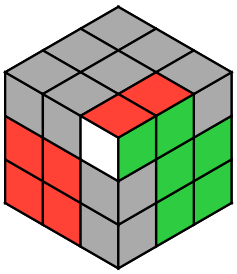
```
#draw_cube(
  apply(
    cube(),
    "M2 E2 S2"
  )
)
```



```
#draw_cube(
  apply(
    cube(size: 4),
    "2R L' 2-3u'"
  )
)
```

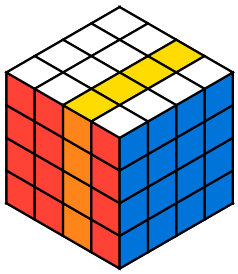


```
#draw_cube(
  apply(
    f2l-cube,
    inverse: true,
    "U R U' R'"
  )
)
```

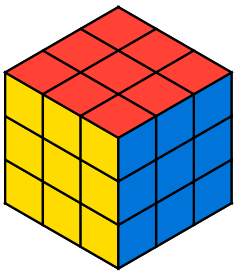


There are also two alternative functions that let you modify the cube: `#rotate_layer` and `#rotate_cube`.

```
#draw_cube(
  rotate_layer(
    cube(size: 4),
    "r",
    depth: 2,
    turns: 2
  )
)
```



```
#draw_cube(
  rotate_cube(
    cube(),
    "x"
  )
)
```

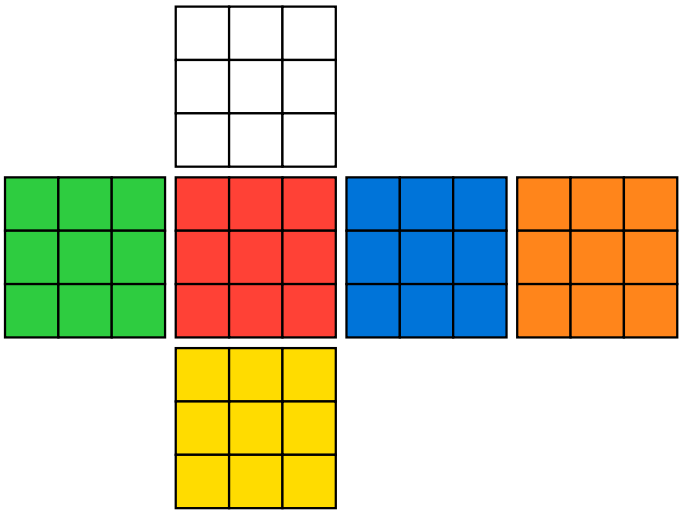


IV.4 Rendering

IV.4.1 Flat view

It is possible to get a full representation of the cube with `#draw_flat`. This function takes a `cube` and draws its unfolded net. It also accepts a `⟨length⟩` argument to change the length of each face edge.

```
#draw_flat(cube())
```



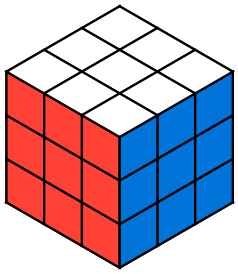
The faces are displayed in the standard cube net layout. The center row contains the left, front, right and back faces. The upper face is placed above the front face, and the down face is placed below it.

IV.4.2 3D view

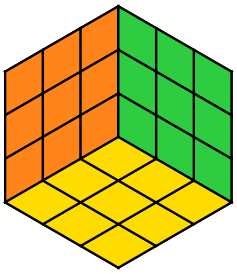
You can also draw a three-dimensional representation of the cube. This is done with `#draw_cube`.

Apart from the `⟨cube⟩` and `⟨length⟩` arguments, which behave the same as in `#draw_flat`, it also accepts `⟨x⟩`, `⟨y⟩` and `⟨z⟩` arguments to customize the cube's orientation. By default, the cube is drawn in an isometric projection.

```
#draw_cube(
  cube(),
)
```



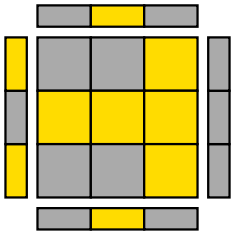
```
#draw_cube(
  cube(),
  x: -35.264deg,
  y: 225deg,
)
```



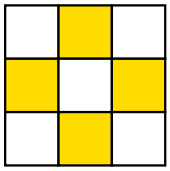
IV.4.3 Face view

The third rendering mode is achieved with `#draw_face`. This draws a single face, optionally including the first row of the adjacent faces. This view is commonly used to illustrate Orientation of the Last Layer (OLL) and Permutation of the Last Layer (PLL) algorithms.

```
#draw_face(
  apply(
    oll-cube,
    "F R U R' U' F'",
    inverse: true
  ),
  "u"
)
```

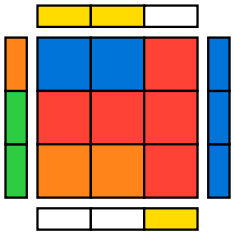


```
#draw_face(
  apply(
    cube(),
    "M2 E2 S2"
  ),
  "u",
  adjacent-faces:
false
)
```

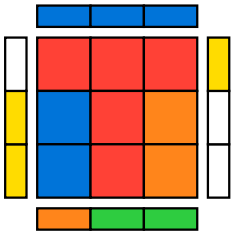


In addition to the face to display, you may also specify the face displayed at the top to control the orientation of the cube. This argument defaults to `auto`, which means that the ⟨`top-face`⟩ will take the value of `"u"` if ⟨`face`⟩ is set to `"f"`, `"r"`, `"b"` or `"l"`; `"f"` if ⟨`face`⟩ is `"d"` and `"b"` if ⟨`face`⟩ is `"u"`.

```
#draw_face(
  apply(
    cube(),
    "F R l U' R2",
    inverse: true
  ),
  "u"
)
```



```
#draw_face(
  apply(
    cube(),
    "F R l U' R2",
    inverse: true
  ),
  "u",
  top-face: "r"
)
```



IV.4.4 Specialized views

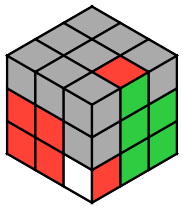
The package also provides three convenience rendering functions. These functions are convenience wrappers around the rendering functions described above. They take an algorithm, applies it to a predefined cube, and displays both the rendered cube and the algorithm.

> Check the API Reference on Section VI for a complete list of arguments.

#draw_f2l

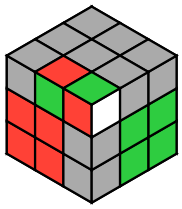
```
#draw_f2l(
  "(R U' R' U)    (R
U' R')",
  length: 45pt

)
```



(R U' R' U)
(R U' R')

```
#draw_f2l(
  "d (R' U2 R) d' (R
U R')",
  length: 45pt
)
```

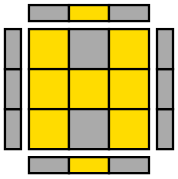


d (R' U2 R) d'
(R U R')

#draw_oll

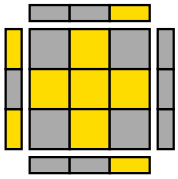
```
#draw_oll(
  "(R U R' U') r R'
(U R U' r')",
  length: 45pt

)
```



(R U R' U') r R'
(U R U' r')

```
#draw_oll(
  "R U2 R2 U' R2 U'
R2 U2 R",
  length: 45pt
)
```



R U2 R2 U' R2 U'
R2 U2 R

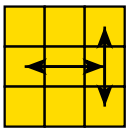
#draw_pll

```
#draw_pll(
  "F R U' R' U' R U
R' F' R U R' U' R' F
R F'",
  length: 45pt

)
```

F R U' R' U' R U
R' F' R U R' U'
R' F R F'

```
#draw_pll(
  "R U R' U' R' F R2
U' R' U' R U R' F'",
  length: 45pt
)
```



R U R' U' R' F
R2 U' R' U' R U
R' F'

Arrows are an experimental feature for `#draw_face` and will improve in a future release.

Part V

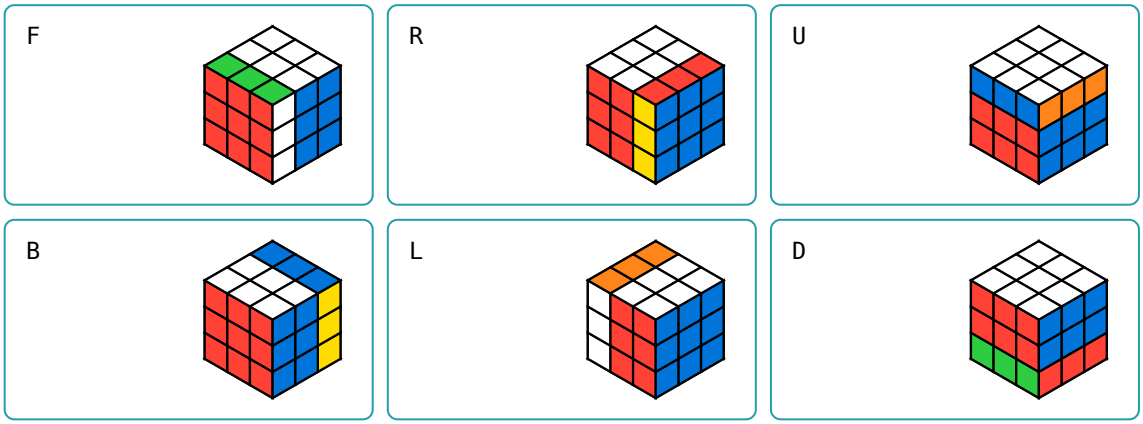
Cube notation

Algorithms are represented as strings, consisting of a sequence of moves separated by spaces. These moves may represent rotations of one or more layers, or even of the entire cube.

Parentheses may also be used for clarification, they are ignored by the parser. Any other character that is not part of the valid notation will cause an error.

V.1 Single-layer moves

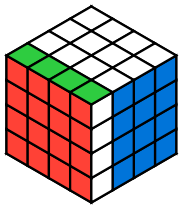
The basic moves are represented with a single uppercase letter. There are six moves, one for each face of the cube: **F** (front), **R** (right), **U** (upper), **B** (back), **L** (left) and **D** (down). Each represents a single clockwise rotation. Double and counterclockwise rotations are explained in Section V.5.



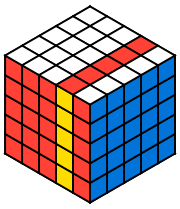
It is also possible to move an inner layer. To do so, prefix the letter with the depth of the layer. The outermost layer has a depth of 1 and layer numbering always starts from the referenced face.

> The maximum depth is one less than the cube size. If you want to rotate the opposite face use the corresponding notation, i.e., in a 4x4x4 cube a 4F rotation is not allowed, the correct notation is B'.

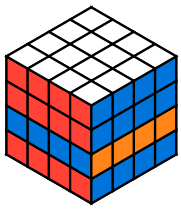
F (or 1F)



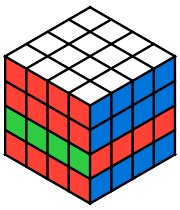
2R



3U



2D



It is not necessary to specify the depth for the moves of the outermost layer. For example, 1U is equivalent to U in all cubes.

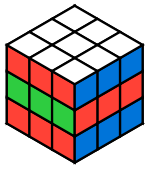
V.2 Central moves

The central layers have a special notation:

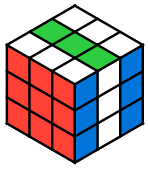
- **M**: “middle”, between L and R following L.
- **E**: “equator”, between U and D following D.
- **S**: “standing”, between F and B following F.

M

E



S



Central moves are only available on cubes with odd size.

V.3 Wide moves

It is also possible to rotate more than one layer at the same time. This is especially useful for big cubes, but it is also used sometimes on a 3x3x3.

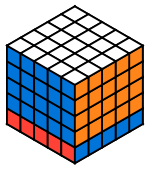
There are two equivalent notations: using lowercase letters or appending a **w** after the corresponding uppercase letter. By default, all layers between the outermost layer and the center (if the cube has odd size) are rotated, but this can be specified with a number before the move. For example, **3f** (or **3Fw**) rotates the first three front layers.

Just as before with the depth parameter, the maximum number of layers that can be moved with a wide move is equal to size - 1. A specific notations exists for rotating the entire cube that will be explained next.

`f / Fw`

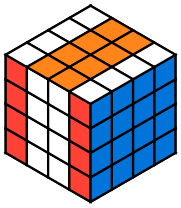
`r / Rw`

`4u / 4Uw`

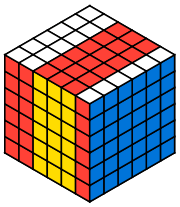


It is also possible to rotate multiple inner layers. To do so, write the first and last layers before the move separated with a dash.

`2-3l / 2-3Lw`



`2-4r / 2-4Rw`

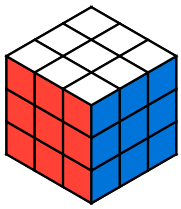


V.4 Cube rotations

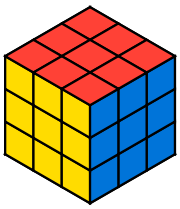
For a complete cube rotation the letters **x**, **y** and **z** are used. These movements do not alter the cube's state, only the viewing orientation.

- **x**: rotating the whole cube as if performing **R**.
- **y**: rotating the whole cube as if performing **U**.
- **z**: rotating the whole cube as if performing **F**.

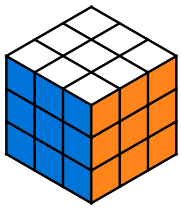
`Original state`



`x`



`y`

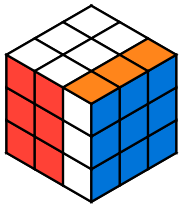


`z`

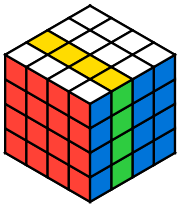
V.5 Modifiers

Appending an apostrophe (') or a "**2**" to a move denotes a counterclockwise rotation or a double one (180º), respectively. These modifiers can be applied to any notation described above.

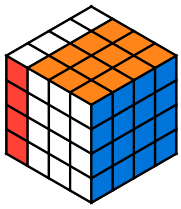
R'



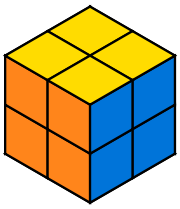
2F2



3Rw'

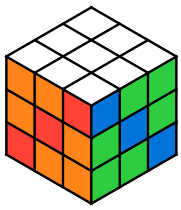


x2

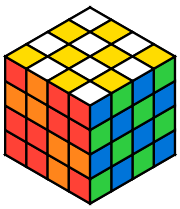


> You can also use parenthesis for improving the readability of the algorithms. They will be ignored by the parser.

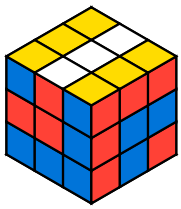
F B R L F B R L F B R L



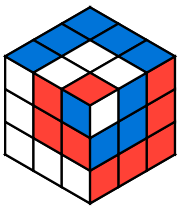
B2 R2 D' F2 D2 B2 U B2 L2 U L' R' B D U L2 R Fw2 Lw2 D U Bw2 Dw2 R2 B U2 R L



F2 R' B' U R' L F' L F' B D' R B L2



U' L' U' F' R2 B' R F U B2 U B' L U' F U R F'



Part VI
API Reference

VI.1 Custom types

VI.1.1 `face`

`face` is the type used throughout the package to refer to one of the six faces of the cube. It accepts the following `str` values:

- `"f"`: for the **f**ront face.
- `"r"`: for the **r**ight face.
- `"u"`: for the **u**pper face.
- `"b"`: for the **b**ack face.
- `"l"`: for the **l**eft face.
- `"d"`: for the **d**own face.

> Face identifiers are lowercase and case-sensitive.

VI.1.2 `cube`

`cube` represents the complete state of a Rubik's cube, it is the main type in this package. However, you should not create or modify instances manually, as functions such as `#draw_cube` require the cube to be in a valid, consistent state. Instead, use `#cube` to create instances and `#apply` to manipulate them.

```
(
  ⟨size⟩ number
  ⟨f⟩ array of color
  ⟨r⟩ array of color
  ⟨u⟩ array of color
  ⟨b⟩ array of color
  ⟨l⟩ array of color
  ⟨d⟩ array of color
)
```

> Each array must contain $\langle \texttt{size} \rangle^2$ elements.

VI.1.3 `face-colors`

This type is used when creating a cube to specify the color of each face.

```
(
  ⟨f⟩ color
  ⟨r⟩ color
  ⟨u⟩ color
  ⟨b⟩ color
  ⟨l⟩ color
  ⟨d⟩ color
)
```

> Not all keys need to be present for a valid `face-colors` value.

VI.1.4 `cube-stickers`

This type is used when creating a cube to specify the color of the stickers for each face. All the arrays must have the same length and it must be equal to the square of the size of the cube.

```
(
  ⟨f⟩ array of color
  ⟨r⟩ array of color
  ⟨u⟩ array of color
  ⟨b⟩ array of color
  ⟨l⟩ array of color
  ⟨d⟩ array of color
)
```

> Not all keys need to be present for a valid `cube-stickers` value.

VI.2 Predefined cubes

#f2l-cube `cube`

3x3x3 cube with only the first 2 layers colored. The upper layer is in gray color.

#oll-cube `cube`

3x3x3 cube with only the upper face colored. The rest of the cube is in gray color.

#solved-cube `cube`

3x3x3 solved cube.

VI.3 Functions

VI.3.1 Cubes

`#cube`

`#cube(⟨size⟩: 3, ⟨colors⟩: auto, ⟨stickers⟩: auto)` → `cube`

This function creates a `cube` given the size, colors, and/or stickers. This is the correct way to create `cube` objects.

The `⟨stickers⟩` argument lets you specify all the stickers on a face. Note that if a face is specified with this argument, it must be fully specified, and the corresponding entry in `⟨colors⟩` is ignored.

> For each face present in `⟨stickers⟩`, the corresponding value in `⟨colors⟩` is ignored.

Argument

`⟨size⟩: 3` — `int`

The size of the cube. The default value is 3, i.e., a standard cube.

Argument

`⟨colors⟩: auto` — `face-colors` | `auto`

The colors assigned to each face.

It must be a `face-colors`, although it does not need to contain all the keys. In that case, the default color value will be used.

The default values for each face are:

```
(
  f: red,
  r: blue,
  u: white,
  b: orange,
  l: green,
  d: yellow
)
```

Argument

`⟨stickers⟩: auto` — `cube-stickers` | `auto`

The stickers of the cube. By default, the cube is initialized in the solved state. See Section IV.2 for more details and examples.

VI.3.2 Algorithms

#apply(⟨cube⟩, ⟨alg⟩, ⟨inverse⟩: false) → cube

Returns a new `cube` with an algorithm applied.

Argument
⟨cube⟩ `cube`

The cube to which the algorithm is applied.

Argument
⟨alg⟩ `str`

A string containing the algorithm to apply, following the notation specified in Section V.

Argument
⟨inverse⟩: false `bool`

If true, the inverse of the algorithm is applied. This is useful when documenting solution algorithms, which are intended to solve the resulting cube.

#rotate_cube(⟨cube⟩, ⟨axis⟩) → cube

Returns a new `cube` with the specified rotation applied.

It returns the cube after applying the rotation. It is used for applying the rotations explained in Section V.4.

Argument
⟨cube⟩ `cube`

The cube to apply the rotation.

Argument
⟨axis⟩ `str`

The axis around which the cube is rotated. Must be one of `("x", "y", "z")`.

#rotate_layer(⟨cube⟩, ⟨face⟩, ⟨depth⟩: 1, ⟨turns⟩: 1) → cube

Returns a new `cube` with the specified layer rotated.

It returns the cube after applying the rotation. It is used for applying the 1-layer rotations explained in Section V.1.

Argument
⟨cube⟩ `cube`

The cube to apply the rotation.

Argument
⟨face⟩ face

The face used to identify the layer to rotate.

Argument
⟨depth⟩: 1 int

The depth of the layer to rotate, by default 1, i.e., the outermost face.

It must be smaller than the cube size.

Argument
⟨turns⟩: 1 int

The number of rotations to apply, by default 1.

VI.3.3 Rendering

#draw_cube	#draw_face	#draw_oll
#draw_f2l	#draw_flat	#draw_pll

```
#draw_cube(
  (cube),
  (x): 35.264deg,
  (y): 45deg,
  (z): 0deg,
  (length): 60pt
) → content
```

Draws a 3D representation of the cube.

The orientation can be modified with the ⟨x⟩, ⟨y⟩ and ⟨z⟩ arguments. The default orientation is an isometric projection. For more details, see the CeTZ documentation[2].

Argument
⟨cube⟩ cube

The cube to draw.

Argument
⟨x⟩: 35.264deg angle

Rotation around the x-axis.

Argument
⟨y⟩: 45deg angle

Rotation around the y-axis.

[2] https://cetz-package.github.io/docs/api/draw-functions/projections/ortho

Argument

`(z): 0deg` `angle`

Rotation around the z-axis.

Argument

`(length): 60pt` `length`

The length of the side of the cube.

> Note that it is different from the cube length. `(length)` specifies the length of **one cube edge before projection**.

```
#draw_f2l((alg), (cube): f2l-cube, (length): 60pt)
```

Draws a cube after applying the algorithm alongside the algorithm. The default cube is `#f2l-cube`.

Argument

`(alg)` `str`

The algorithm to apply.

Argument

`(cube): f2l-cube` `cube`

The cube to draw.

Argument

`(length): 60pt` `length`

The length of the side of the cube. See `#draw_cube`.

```
#draw_face(
  (cube),
  (face),
  (top-face): auto,
  (length): 60pt,
  (adjacent-faces): true
) → content
```

Draws a single face of the cube, and optionally the first row of the adjacent faces.

Argument

`(cube)` `cube`

The cube to draw.

Argument

`(face)` `face`

The face to display.

It must be one of `("f", "r", "u", "b", "l", "d")`.

Argument

`⟨top-face⟩: auto` `face` | `auto`

The face that will be displayed above the main face. It defines the orientation of the cube. If set to `auto`, the logical default is used:

- For `"f"`, `"r"`, `"b"` and `"l"`: `"u"`.
- For `"u"`: `"b"`.
- For `"d"`: `"f"`.

Valid values are `auto` and any `face` except the selected in `⟨face⟩` and its opposite.

Argument

`⟨length⟩: 60pt` `length`

The length of the side of the cube.

Argument

`⟨adjacent-faces⟩: true` `bool`

Whether the adjacent faces are displayed.

`#draw_flat(⟨cube⟩, ⟨length⟩: 60pt)` → `content`

Draws a flat representation of the cube.

Argument

`⟨cube⟩` `cube`

The cube to draw.

Argument

`⟨length⟩: 60pt` `length`

The length of the side of the cube.

`#draw_oll(⟨alg⟩, ⟨cube⟩: oll-cube, ⟨length⟩: 60pt)`

Draws a face after applying the algorithm alongside the algorithm. The default cube is `#oll-cube`.

Argument

`⟨alg⟩` `str`

The algorithm to apply.

Argument

`⟨cube⟩: oll-cube` `cube`

The cube to draw.

Argument

(length): 60pt `length`

The length of the side of the cube. See #draw_cube.

```
#draw_pll(
  (alg),
  (cube): solved-cube,
  (adjacent-faces): false,
  (arrows): true,
  (length): 60pt
)
```

Draws a face after applying the algorithm alongside the algorithm. It may also show arrows with the movement of the pieces on the upper face. The default cube is #oll-cube.

Argument

(alg) `str`

The algorithm to apply.

Argument

(cube): solved-cube `cube`

The cube to draw.

Argument

(adjacent-faces): false `bool`

Whether to show or not the adjacent faces. See #draw_face

Argument

(arrows): true `bool`

Whether to display arrows representing the moves.

Argument

(length): 60pt `length`

The length of the side of the cube. See #draw_cube.

Part VII

Index

A

C

D

F

O

R

S